GUIDE

THE SOVEREIGN AI STACK BLUEPRINT

Your Complete Guide to Owning Your Intelligence: From Zero to DGX

By PhantomByte

WHAT YOU WILL LEARN

- Reuse the laptop you already own ($0)
- Build a budget beast for under $1,200
- Assemble an enthusiast workstation that laughs at cloud bills
- Buy a pre-built Windows RTX AI PC or Mac Studio
- Deploy an NVIDIA DGX for team-scale inference
- Run Apple Silicon for silent, efficient local AI

Every prompt you send to ChatGPT, Claude, or Gemini becomes someone else's data.

They train on it. They moderate it. They can revoke your access tomorrow.

Sovereign AI means your models run on your metal. Your data never leaves your network. There are no rate limits, no subscription bills, and no black-box updates that lobotomize your assistant overnight.

This blueprint covers EVERY path to sovereignty:

- Reuse the laptop you already own ($0)
- Build a budget beast for under $1,200
- Assemble an enthusiast workstation that laughs at cloud bills
- Buy a pre-built Windows RTX AI PC or Mac Studio
- Deploy an NVIDIA DGX for team-scale inference
- Run Apple Silicon for silent, efficient local AI

Pick your path. Let's build.

PART 1: HARDWARE – THE COMPLETE OPTIONS MATRIX

We break this into six tiers, from "use what you have" to "datacenter-grade."

Each tier lists exactly what you can run, how fast, and what it costs.

TIER 0: THE "$0 REUSE WHAT YOU OWN" BUILD

Use any computer you already have. A gaming PC, a MacBook, an old workstation – Ollama runs on damn near everything.

MINIMUM SPECS:

CPU: Any quad-core from the last 8 years (Intel i5-8400 or better)
RAM: 16 GB DDR4 (8 GB works for 3B models, but 16 is the floor)

Disk: 50 GB free SSD space (model files are huge)

GPU: Optional – CPU inference handles 7B-8B models

WHAT YOU CAN RUN:

- Qwen 2.5 7B / Llama 3.2 3B / Gemma 3 4B – CPU only, 5-15 tokens/sec
- DeepSeek-R1 7B/8B – CPU only, usable for chat + reasoning
- Embedding models (nomic-embed-text, mxbai-embed-large)
- Small code models (Qwen 2.5 Coder 7B)

TOTAL COST: $0. You already own the machine.

TIER 1: THE "BUDGET SOVEREIGNTY" BUILD (~$900 – $1,200)

Purpose-built for local AI. This is the sweet spot. It runs 8B-14B models comfortably and stretches to 32B quantized models.

CPU: AMD Ryzen 5 7600X (6-core, 5.3 GHz, AM5)
$200 | AM5 platform gives you an upgrade path.
Alternative: Intel i5-13600K (~$280) if you prefer Team Blue.
GPU: NVIDIA RTX 4060 Ti 16 GB VRAM
$430–$480 | 16 GB VRAM is the magic number. It fits 13B-14B models fully in GPU memory at 4-bit quantization.

Alternative: Used RTX 3090 24 GB (~$600–$700 used). Extra VRAM opens up 32B models. Best value move if you are comfortable with used hardware.

MOBO: B650 chipset (AM5) – ASRock B650M Pro RS or MSI B650 Tomahawk
$140–$180 | PCIe 5.0 ready, 4 RAM slots for future expansion.

RAM: 32 GB (2x16 GB) DDR5-6000 CL30
$95–$110 | When a model does not fit in VRAM, it spills to system RAM. DDR5-6000 keeps that spillover fast.

STORAGE: 1 TB NVMe SSD (Gen 4)
$60–$75 | Llama 3.1 70B 4-bit = 40 GB. A few models and 500 GB vanishes. Samsung 990 EVO or WD Black SN770.

PSU: 750W Gold (Corsair RM750e or Thermaltake GF1)
$90–$100 | Headroom for a second GPU later.

CASE: Any ATX mid-tower with decent airflow ($60–$80)
Fractal Pop Air, Corsair 4000D, NZXT H5 Flow.

COOLER: Thermalright Peerless Assassin 120 SE ($35)
Beats coolers that cost 3x. Air cooling is fine for a 105W CPU.

TOTAL: ~$930–$1,160

WHAT YOU CAN RUN:

- Llama 3.1 8B – GPU only, 80+ tokens/sec
- Qwen 2.5 14B – GPU only at 4-bit, 50+ tokens/sec
- DeepSeek-R1 14B – GPU only, reasoning runs
- Gemma 3 12B – Vision + text on GPU
- Qwen 2.5 Coder 14B – Full GPU code generation
- Llama 3.1 70B 4-bit – GPU + CPU offload, 5-8 tokens/sec (usable)
- Any embedding model – instant

TIER 2: THE "ENTHUSIAST" BUILD (~$2,500 – $3,500)

The build that makes cloud subscriptions look stupid. Runs 32B-70B models at interactive speeds. Dual GPU ready.

CPU: AMD Ryzen 9 7950X (16-core, 5.7 GHz) – $520
Or: Intel i9-14900K (24-core) – $550

GPU – PICK ONE PATH:

PATH A – Single GPU:

NVIDIA RTX 4090 24 GB – $1,600–$1,800

Best single consumer GPU. 24 GB runs 34B at 4-bit entirely in VRAM.

PATH B – Dual GPU (better value):

2x Used RTX 3090 24 GB – $1,200–$1,400 total

48 GB combined VRAM. Runs 70B models comfortably.

Requires a bigger PSU and case with dual GPU support.

MOBO: X670E chipset (dual GPU PCIe lanes)

ASRock X670E Steel Legend or MSI X670E Tomahawk – $260–$300

RAM: 64 GB (2x32 GB) DDR5-6000 CL30 – $180–$210

64 GB means you never worry about spillover.

STORAGE: 2 TB NVMe SSD Gen 4 – $110–$130

Samsung 990 Pro or WD Black SN850X.

PSU: 1200W Platinum (Seasonic Prime or Corsair HX1200) – $230–$280

Dual 3090s pull 700W+ under load. Do not cheap out here.

CASE: Fractal Meshify 2 XL or Lian Li O11 Dynamic – $150–$180

COOLER: Arctic Liquid Freezer III 360mm AIO – $120

TOTAL: ~$2,500–$3,500 (depending on GPU path)

WHAT YOU CAN RUN:

- Qwen 3 32B – GPU only at 4-bit, 40+ tokens/sec
- Llama 3.1 70B 4-bit – Full GPU with dual 3090s, 20+ tokens/sec
- DeepSeek-R1 70B – Dual GPU, full reasoning at interactive speed
- Gemma 4 26B – Single GPU, vision + text
- Qwen 2.5 Coder 32B – Production-grade code gen on GPU
- GPT-OSS 20B – Full GPU, OpenAI open-weight

TIER 3: THE "F*CK THE CLOUD" WORKSTATION (~$6,000 – $10,000)

The build you get when you have already saved $900/month on cloud bills and you are reinvesting. Runs 70B-120B models. Multi-GPU, server-grade optional.

CPU: AMD Threadripper 7960X (24-core, PCIe 5.0, 48 lanes) – $1,500

Or: AMD Ryzen 9 9950X (16-core, cheaper platform)

Threadripper if you need PCIe lanes for 3-4 GPUs. 9950X otherwise.

GPU: 2x-4x RTX 4090 24 GB or RTX 5090 (when available)

2x 4090: 48 GB VRAM – $3,200–$3,600

4x 4090: 96 GB VRAM – $6,400–$7,200

Alternative: Used A6000 48 GB workstation cards ($2,500 each used)

for denser VRAM in fewer slots.

MOBO: TRX50 (Threadripper) or X870E (AM5)

TRX50: ASRock TRX50 WS – $900

X870E: ASRock X870E Taichi – $450

RAM: 128 GB DDR5 (4x32 GB) – $350–$400

Or: 192 GB (4x48 GB) if you want to run MoE models with large spillover.

STORAGE: 4 TB NVMe Gen 4 (primary) + 8 TB HDD (model archive)

SSD: $250–$300 | HDD: $140

PSU: 1600W+ (dual PSU or server PSU)

Super Flower Leadex 1600W or dual Corsair HX1500i.

CASE: Server chassis or open-air mining frame

Mining frames are ugly but practical for 4 GPUs: $80

Phanteks Enthoo Pro 2 Server Edition: $180

COOLING: GPU water blocks + custom loop or run fans loud

Custom loop: +$500–$800. Air cooling 4 GPUs: undervolt them.

TOTAL: $6,000–$10,000+

WHAT YOU CAN RUN:

- Llama 3.1 405B 4-bit (if you have enough VRAM pooled)
- DeepSeek-R1 671B (MoE – only active parameters loaded)
- Qwen 3.5 122B – Multi-GPU, near-frontier performance
- GPT-OSS 120B – Full multi-GPU
- Fine-tune 7B-14B models locally with QLoRA
- Run multiple models simultaneously (chat + embedding + code gen)
- Anything on Ollama's library – you have the VRAM

TIER 4: WINDOWS RTX AI PC / RTX SPARK (Pre-Built, ~$1,500 – $4,000)

Not everyone wants to build a PC. NVIDIA and partners now ship pre-built RTX AI PCs and the RTX Spark (Project DIGITS) category – compact boxes with laptop-class RTX GPUs, tuned for local inference out of the box.

WHAT IS IT:

- Pre-built desktops and SFF systems with RTX 4060/4070/4080/4090 Laptop

or Desktop GPUs

- Ships with Windows 11 + AI-optimized drivers
- Often bundled with NVIDIA AI Workbench, Ollama, and local LLM tools
- Some models include NVIDIA RTX 5000 Ada workstation GPUs

POPULAR CONFIGURATIONS:

MSI / ASUS / Alienware RTX AI PC (RTX 4070 Ti Super 16 GB)

~$1,800–$2,400

Plug-and-play. 16 GB VRAM handles 14B models on GPU.

Good for: developers who want zero build time.

NVIDIA RTX Spark / Project DIGITS style SFF

~$1,500–$3,000 (depending on GPU tier)

Compact, quiet, designed for AI inference on a desk.

Good for: creators, small offices, anyone who wants a clean setup.

HP Z / Dell Precision Workstation (RTX 5000 Ada 32 GB)

~$3,500–$5,000

ISV-certified, rock-solid drivers, 32 GB VRAM.

Good for: professionals who need stability over raw cost savings.

PROS:

- Zero assembly. Plug in, install Ollama, pull models, go.
- Warranty and support.
- Compact and quiet compared to custom enthusiast builds.

CONS:

- More expensive per teraflop than DIY.
- Less upgrade flexibility (laptop GPUs soldered in some SFF units).
- Windows adds friction for headless/server workflows.

BEST FOR: Buyers who value time over money, corporate environments, or anyone who wants a clean desk and a support number.

TIER 5: APPLE SILICON – MAC STUDIO / MAC PRO / MACBOOK PRO (~$2,000 – $8,000)

Apple Silicon is the dark horse of sovereign AI. No CUDA. No NVIDIA drivers.

But Apple's unified memory architecture is exceptional for inference,

and the M-series chips have dedicated Neural Engine silicon.

HARDWARE OPTIONS:

MacBook Pro (M3 Pro / M3 Max)

M3 Pro: 18 GB – 36 GB unified memory – $2,000–$3,200

M3 Max: 36 GB – 128 GB unified memory – $3,500–$5,000

Portable. Silent. Runs Ollama natively on Metal.

Mac Studio (M2 Ultra / M3 Ultra)

M2 Ultra: 64 GB – 192 GB unified memory – $4,000–$6,000

M3 Ultra: 256 GB unified memory option – ~$8,000+

Desktop power. No fan noise. Runs models that need 100+ GB RAM.

Mac Pro (M2 Ultra)

64 GB – 192 GB unified memory – $7,000–$10,000+

Overkill for most. Buy only if you also need Thunderbolt expansion.

WHAT YOU CAN RUN ON APPLE SILICON:

- Llama 3.1 8B / Qwen 2.5 7B – M3 Pro, 30–50 tokens/sec
- Llama 3.1 70B 4-bit – M2 Ultra 192 GB, entirely in unified memory
- Qwen 3 32B – M3 Max 128 GB, fast interactive speeds
- DeepSeek-R1 32B – M2 Ultra, fully resident in memory
- Embedding models – instant on Neural Engine
- Fine-tuning with MLX (Apple's ML framework) – supported natively

PROS:

- Unified memory means no VRAM bottleneck. A 192 GB Mac Studio loads a

70B model like it is nothing.

- Silent operation. No GPU fans screaming at 3 AM.
- Ollama runs natively on Metal – no CUDA driver hell.
- MLX ecosystem growing fast for training and fine-tuning.

CONS:

- No multi-GPU scaling. One machine, one pool of memory.
- No CUDA = no PyTorch-native training at NVIDIA speeds.
- Premium price per teraflop compared to used NVIDIA cards.
- No upgrade path. You cannot swap the chip for a faster one later.

BEST FOR: Developers who prioritize silence, simplicity, and portability.

Also ideal if you already live in the Apple ecosystem.

TIER 6: NVIDIA DGX – THE ENTERPRISE SOVEREIGN PLAY ($40,000 – $200,000+)

The NVIDIA DGX is not a PC. It is a purpose-built AI supercomputer.

If you are a team, a lab, or a company that wants on-premise frontier models without sending data to OpenAI, this is the apex predator.

CURRENT LINEUP:

NVIDIA DGX Station (A100 or H100)

4x–8x A100 40 GB or 80 GB GPUs

320 GB – 640 GB total VRAM

~$150,000–$250,000

Water-cooled tower. Fits under a desk. Powers a research lab.

NVIDIA DGX A100 / H100 Server

8x A100 or H100 SXM modules

640 GB – 1,280 GB total VRAM (with NVLink)

~$200,000–$500,000+

Rackmount. Datacenter power. Runs GPT-4-class models locally.

NVIDIA DGX B200 (Blackwell generation)

8x B200 GPUs

~1,440 GB unified VRAM with NVLink

~$400,000+

The absolute frontier. Local deployment of models at 100B+ scale.

WHAT YOU CAN RUN:

- Literally anything. 70B, 120B, 405B, even full-parameter dense models.

- DeepSeek-R1 671B MoE – active parameters fit easily.
- Multi-user concurrent inference for a team of 50+.
- Full fine-tuning (not just QLoRA) of 30B+ models.
- Training from scratch on proprietary datasets.

PROS:

- Enterprise support from NVIDIA.
- NVLink means GPUs act as one giant memory pool.
- Optimized software stack (Base Command, AI Enterprise).
- No cloud egress fees, no API rate limits, no data leakage.

CONS:

- Price. Obviously.
- Power and cooling requirements (datacenter-grade).
- Overkill for a single user.

BEST FOR: Teams, research labs, fintech, healthcare, defense, and any organization that cannot send data to third-party APIs.

TIER 7: CLOUD GPU RENTAL – THE HYBRID PATH ($0.50 – $5.00 / hour)

Sometimes you want sovereignty without the CapEx. Rent a GPU instance, load your models, and treat it like your own server.

PROVIDERS:

- Lambda Labs – RTX A6000, A100, H100 hourly rental
- RunPod – RTX 4090, A6000, A100 pods
- Vast.ai – Peer-to-peer GPU rental (cheapest, variable reliability)
- CoreWeave – Enterprise GPU cloud, NVIDIA-partnered
- Google Cloud / AWS / Azure – A100/H100 spot instances

COST:

- RTX 4090 equivalent: ~$0.50–$1.50 / hour
- A100 40 GB: ~$1.50–$3.00 / hour
- H100 80 GB: ~$3.00–$5.00 / hour

SOVEREIGNTY LEVEL: Medium. You own the model weights and the data pipeline,

but you do not own the hardware. Fine for experimentation and burst workloads.

Not true sovereignty, but a viable stepping stone.

PART 2: THE SOFTWARE STACK

Layer by layer. Install in this order.

LAYER 1: OPERATING SYSTEM

Ubuntu 24.04 LTS (RECOMMENDED)

Best CUDA support. Headless-friendly. Docker works flawlessly.
If you are serious, use Linux.

Windows 11 + WSL2

Works. Docker and GPU passthrough add friction, but it is manageable.
Good for: gamers who want to dual-use their machine, or corporate environments locked to Windows.

macOS (Apple Silicon)

Ollama runs native on Metal. No NVIDIA CUDA, but Apple's unified memory is excellent for inference. Skip if you need multi-GPU or heavy training.

QUICK START (Ubuntu):

sudo apt update && sudo apt upgrade -y

sudo apt install build-essential curl git htop nvtop -y

LAYER 2: NVIDIA DRIVERS + CUDA (Skip on Mac)

Ubuntu 24.04 – Install NVIDIA driver

sudo apt install nvidia-driver-550 -y

sudo reboot

Verify after reboot

nvidia-smi

Should show: Driver Version 550.x, CUDA Version 12.4

Install CUDA toolkit (for custom builds, not strictly required)

wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda-keyring_1.1-1_all.deb

sudo dpkg -i cuda-keyring_1.1-1_all.deb

sudo apt update

sudo apt install cuda-toolkit-12-4 -y

MAC USERS: Skip this layer. Ollama uses Metal automatically.

LAYER 3: OLLAMA – THE MODEL SERVER

Linux / WSL2

curl -fsSL https://ollama.com/install.sh | sh

macOS – Download from https://ollama.com/download

Verify

ollama --version

ollama serve # Starts the server (localhost:11434)

Pull your first models (start with these)

ollama pull qwen3:14b # Best all-around for 16 GB VRAM

ollama pull llama3.1:8b # Reliable workhorse

ollama pull nomic-embed-text # Embeddings for RAG

ollama pull qwen2.5-coder:14b # Code generation

ollama pull deepseek-r1:14b # Reasoning tasks

Key environment variables (add to ~/.bashrc or ~/.zshrc)

export OLLAMA_HOST=0.0.0.0 # Expose on LAN (optional)

export OLLAMA_NUM_PARALLEL=4 # Concurrent requests

export OLLAMA_KEEP_ALIVE=24h # Keep models in memory

export OLLAMA_MAX_LOADED_MODELS=4 # Max simultaneous models

For GPU offloading on large models (e.g., 70B on 16 GB VRAM)

Ollama auto-offloads. Set num_gpu layers manually if needed:

ollama run llama3.1:70b

Then /set parameter num_gpu 40

LAYER 4: OPEN WEBUI – THE CHAT INTERFACE

```
# Docker method (recommended – clean isolation)
docker run -d \
-p 3000:8080 \
--add-host=host.docker.internal:host-gateway \
-v open-webui:/app/backend/data \
--name open-webui \
--restart always \
ghcr.io/open-webui/open-webui:main
# Access at http://localhost:3000
# First run: create admin account
# Settings → Connections → Ollama API: http://host.docker.internal:11434
```

FEATURES YOU GET:

- ChatGPT-style interface for local models
- RAG (upload documents, chat with them)
- Web search integration
- Model management UI
- Multi-user support (give family/team access)
- Mobile-friendly PWA

ALTERNATIVE (no Docker):

```
pip install open-webui
open-webui serve
```

LAYER 5: OPENCLAW – AGENT ORCHESTRATION

```
# Install globally via npm
npm install -g openclaw
# Initialize
openclaw init
# Creates ~/.openclaw/ config directory
# Configure for local Ollama (edit ~/.openclaw/openclaw.json)
{
"models": {
```

"primary": "ollama/qwen3:14b",

"fast": "ollama/llama3.1:8b",

"reasoning": "ollama/deepseek-r1:14b"

},

"tools": {

"browser": true,

"exec": true,

"memory": true

}

}

Start OpenClaw Gateway

openclaw gateway start

KEY CAPABILITIES WITH LOCAL MODELS:

- Multi-agent swarms running on your hardware
- Cron jobs for automated tasks (news scraping, monitoring)
- Memory persistence across sessions
- Browser automation for web research
- Telegram / WhatsApp integration

CRITICAL: OpenClaw Gateway + Ollama together = a self-hosted AI assistant that rivals ChatGPT but runs entirely on your machine. Your data never leaves your network.

LAYER 6: HERMES AGENT + CUSTOM AGENTS

Hermes Agent is a general-purpose agent framework designed for local model execution. Use it when OpenClaw's built-in patterns are not enough.

Clone and install

git clone https://github.com/NousResearch/Hermes-Function-Calling.git

cd Hermes-Function-Calling

pip install -r requirements.txt

Or use the Ollama-native approach with Hermes models

ollama pull hermes3:8b # Function-calling model

ollama pull command-r-plus:104b # Cohere's agentic model

AGENT ARCHITECTURE PATTERN (local stack):

Ollama (model serving)

↓

OpenClaw Gateway (orchestration + tools)

↓

Hermes Agent / Custom Python agents (specialized logic)

↓

Open WebUI (user interface)

EXAMPLE: Local RAG agent that searches your documents

1. Ollama serves nomic-embed-text + qwen3:14b
2. OpenClaw handles document ingestion + search trigger
3. Custom Python agent does chunking + vector search
4. Results surface in Open WebUI chat

LAYER 7: OPTIONAL – BUT HIGHLY RECOMMENDED

A) n8n – Workflow Automation

docker run -d --name n8n -p 5678:5678 \

-v n8n_data:/home/node/.n8n \

--restart always \

n8nio/n8n

Access at http://localhost:5678

Connect to Ollama via HTTP Request nodes → localhost:11434

B) Continue.dev or Cline – IDE Integration

VS Code Extension: Continue

Configure to use Ollama:

{

"models": [{

"title": "Qwen 14B",

"provider": "ollama",

"model": "qwen3:14b"

}]

}

C) Anything LLM – Document RAG Platform

docker run -d -p 3001:3001 \

-v anythingllm:/app/server/storage \

mintplexlabs/anythingllm

D) pgvector – Vector Database for RAG

docker run -d --name pgvector -p 5432:5432 \

-e POSTGRES_PASSWORD=yourpassword \

-v pgvector_data:/var/lib/postgresql/data \

pgvector/pgvector:pg16

E) Docker Compose – The One-File Stack

Save as docker-compose.yml, run: docker compose up -d

version: '3.8'

services:

```
  ollama:
    image: ollama/ollama:latest
    ports: ["11434:11434"]
    volumes:
      - ollama_data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    ports: ["3000:8080"]
    volumes:
      - webui_data:/app/backend/data
    extra_hosts:
      - "host.docker.internal:host-gateway"
    restart: always
  n8n:
    image: n8nio/n8n
    ports: ["5678:5678"]
    volumes:
      - n8n_data:/home/node/.n8n
    restart: always
```

```
  volumes:
    ollama_data:
    webui_data:
    n8n_data:
```

One command. Your entire stack is running.

PART 3: THE CONFIGURATION CHECKLIST

Follow this in order. Check off each box. When you are done, you have a production-ready sovereign AI stack.

PHASE 1: BARE METAL (30–60 minutes)

- ☑ Assemble hardware (or verify existing machine meets specs)
- ☑ Install Ubuntu 24.04 LTS (or verify existing OS)
- ☑ Run `sudo apt update && sudo apt upgrade -y`
- ☑ Install NVIDIA drivers: `sudo apt install nvidia-driver-550 -y`
- ☑ Reboot, verify with `nvidia-smi`
- ☑ Install Docker: `sudo apt install docker.io -y`
- ☑ Add user to docker group: `sudo usermod -aG docker $USER`
- ☑ Log out and back in (docker group takes effect)

PHASE 2: CORE SERVICES (15–30 minutes)

- ☑ Install Ollama: `curl -fsSL https://ollama.com/install.sh | sh`
- ☑ Pull base models:

- `ollama pull qwen3:14b`
- `ollama pull llama3.1:8b`
- `ollama pull nomic-embed-text`

- ☑ Verify: `curl http://localhost:11434/api/tags`
- ☑ Set environment variables in ~/.bashrc
- ☑ Start Ollama serve (auto-starts via systemd, but verify)
- ☑ Install Open WebUI via Docker
- ☑ Access http://localhost:3000, create admin account
- ☑ Connect Open WebUI to Ollama (Settings → Connections)

PHASE 3: AGENT INFRASTRUCTURE (15–20 minutes)

- ☑ Install Node.js 22+:

`curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -`

`sudo apt install nodejs -y`

- ☑ Install OpenClaw: `npm install -g openclaw`
- ☑ Run `openclaw init`
- ☑ Edit ~/.openclaw/openclaw.json – set models to local Ollama
- ☑ Run `openclaw gateway start`
- ☑ Verify gateway: `openclaw gateway status`
- ☑ Configure Telegram bot (optional) for mobile access
- ☑ Install Python agent dependencies:

`pip install openai httpx numpy`

PHASE 4: HARDENING & OPTIMIZATION (10–15 minutes)

- ☑ Firewall: `sudo ufw enable`

`sudo ufw allow 22/tcp` (SSH)

`sudo ufw allow 3000/tcp` (Open WebUI, restrict to LAN if possible)

`sudo ufw allow 11434/tcp` (Ollama – restrict to localhost)

- ☑ Limit Ollama to localhost:

`sudo systemctl edit ollama.service`

Add: Environment="OLLAMA_HOST=127.0.0.1"

- ☑ Set up nginx reverse proxy for HTTPS (optional, remote access):

`sudo apt install nginx certbot python3-certbot-nginx -y`

- ☑ Configure GPU power limits for 24/7 operation:

`sudo nvidia-smi -pl 250`

- ☑ Set up model auto-loading on boot

PHASE 5: VALIDATION (5 minutes)

- ☑ Test chat: Open http://localhost:3000, ask a question
- ☑ Test code gen: "Write a Python function that calculates SHA-256 hashes"
- ☑ Test RAG: Upload a PDF in Open WebUI, ask about its content
- ☑ Test API:

`curl http://localhost:11434/api/generate -d '{"model":"qwen3:14b","prompt":"Hello"}'`

- ☑ Test OpenClaw via Telegram (if configured)
- ☑ Check VRAM: `nvidia-smi` – model should be loaded

☑ Check performance: `ollama run qwen3:14b --verbose` and note tokens/sec

POST-SETUP: MODEL EXPANSION PLAN

WEEK 1 (Core Models):

- ☑ `ollama pull deepseek-r1:14b` (Reasoning)
- ☑ `ollama pull qwen2.5-coder:14b` (Code generation)
- ☑ `ollama pull gemma3:12b` (Vision + fast chat)

WEEK 2 (Specialized):

- ☑ `ollama pull llama3.1:70b` (Heavy lifting, if VRAM allows)
- ☑ `ollama pull mxbai-embed-large` (Better embeddings for RAG)
- ☑ `ollama pull mistral:7b` (Fast, light alternative)

WEEK 3 (Frontier):

- ☑ `ollama pull qwen3.5:27b` (Near-frontier performance)
- ☑ `ollama pull deepseek-r1:32b` (Deep reasoning on bigger hardware)
- ☑ `ollama pull gemma4:26b` (Google's latest)

PART 4: REAL NUMBERS – CLOUD VS. LOCAL

CLOUD STACK (monthly):

ChatGPT Pro: $200

Claude Pro: $20

API usage (OpenAI): $150–$300

Cursor Pro: $20

Perplexity Pro: $20

Misc (Pinecone, etc): $50–$100

MONTHLY TOTAL: $460–$660+

LOCAL STACK (one-time, amortized over 12 months):

Tier 2 hardware: $1,100 (one-time → $92/month over 12 months)

Electricity (24/7): $15–$25/month

Models: $0

API calls: $0

No rate limits: Priceless

MONTHLY EQUIVALENT: ~$110–$120

ANNUAL SAVINGS: $4,200–$6,500. The hardware pays for itself in 2–3 months.

AND YOU GET:

- Zero data leaving your network
- No censorship or content filtering
- No rate limits – run agents 24/7
- No API key management
- No vendor lock-in
- Models keep working even if OpenAI goes down
- Fine-tune your own models on your own data

PART 5: TROUBLESHOOTING QUICK REFERENCE

PROBLEM: "Ollama says CUDA error / no GPU detected"

FIX: `nvidia-smi` – if that works, restart Ollama:

```
sudo systemctl restart ollama
```

If nvidia-smi fails: reinstall drivers.

PROBLEM: "Model loads but runs on CPU only (slow)"

FIX: Check `ollama ps` while running – shows GPU vs CPU layers.

Small models should show 100% GPU.

If GPU layers = 0: check CUDA toolkit.

If partial GPU: normal for large models. Adjust with /set parameter num_gpu.

PROBLEM: "Out of memory errors"

FIX: Pull a smaller quant: `ollama pull qwen3:14b-q4_K_M`

Or close other models: `ollama stop <name>`

Or reduce context: /set parameter num_ctx 4096

PROBLEM: "Open WebUI can't connect to Ollama"

FIX: In Docker: use http://host.docker.internal:11434

Not in Docker: use http://localhost:11434

Check: `curl http://localhost:11434/api/tags` returns JSON

PROBLEM: "Models are slow (low tokens/sec)"

FIX: Check power management: `nvidia-smi -q -d POWER`

GPU might be throttling. Set: `sudo nvidia-smi -pl 300`

Check temps: `nvidia-smi -q -d TEMPERATURE`

Above 85C: improve airflow, undervolt GPU.

PROBLEM: "Disk space disappearing"

FIX: Ollama stores models in ~/.ollama/models/

`ollama list` shows sizes.

Remove unused: `ollama rm <name>`

Check Docker: `docker system df`

PART 6: THE "NEXT LEVEL" – WHAT TO BUILD AFTER SETUP

Once your stack is running, here is what you can build on it:

1. PERSONAL AI ASSISTANT (OpenClaw + Ollama)

   Telegram bot that knows your calendar, email, files.

   Runs on your hardware. Sees everything you allow.

2. CODE REVIEW AGENT (Qwen 2.5 Coder + git hooks)

   Auto-review every commit against your codebase.

   Catches bugs before they hit production.

3. CONTENT PIPELINE (OpenClaw swarm + Ollama)

   Multi-agent research → drafting → editing pipeline.

   Generates articles, social posts, newsletters.

4. RAG KNOWLEDGE BASE (nomic-embed + pgvector + AnythingLLM)

   Ingest your company docs, wikis, codebases.

   Chat with your entire knowledge base.

5. SECURITY MONITORING AGENT

   Scan logs, detect anomalies, alert you.

Runs 24/7 on spare GPU cycles.

6. FINE-TUNING RIG (QLoRA on your hardware)

Fine-tune 7B-14B models on your own writing style.

Custom models for your specific domain.

7. TEAM AI SERVER (DGX or multi-GPU workstation)

Centralized inference for a whole company.

Role-based access, audit logs, shared model pools.

PART 7: PLATFORM-SPECIFIC NOTES

WINDOWS RTX AI PC / RTX SPARK:

- Install Ollama for Windows from https://ollama.com/download
- Use Docker Desktop with WSL2 backend for Open WebUI and n8n
- GPU passthrough in Docker Desktop: Settings → Resources → WSL Integration
- PowerShell commands work, but Git Bash or Windows Terminal recommended
- NVIDIA AI Workbench can pre-configure containers for you

APPLE SILICON (M-series):

- Install Ollama for Mac. It auto-detects Metal.
- Use Docker Desktop for Apple Silicon (arm64 images).
- For fine-tuning, use MLX (pip install mlx mlx-lm).
- Homebrew is your friend: brew install ollama (if available) or use the .dmg.
- No CUDA = no PyTorch CUDA training. Use MLX or Core ML instead.

NVIDIA DGX:

- Comes with Base Command Manager pre-installed.
- Use NGC (NVIDIA GPU Cloud) containers for PyTorch, TensorFlow, Ollama.
- NVLink means multi-GPU models run as one unified memory pool.
- NVIDIA AI Enterprise license adds support and security patches.

LINUX (ALL NVIDIA BUILDS):

- The reference platform. Everything in this guide assumes Linux first.
- Use nvtop for real-time GPU monitoring (better than nvidia-smi for visuals).

- Enable persistence mode: `sudo nvidia-smi -pm 1` for faster model loading.

RESOURCES & LINKS

Ollama: https://ollama.com

Ollama Models: https://ollama.com/library

Open WebUI: https://github.com/open-webui/open-webui

OpenClaw: https://docs.openclaw.ai

n8n: https://n8n.io

pgvector: https://github.com/pgvector/pgvector

Continue.dev: https://continue.dev

AnythingLLM: https://anythingllm.com

Hermes Agent: https://github.com/NousResearch/Hermes-Function-Calling

NVIDIA DGX: https://www.nvidia.com/dgx

NVIDIA RTX AI PCs: https://www.nvidia.com/en-us/geforce/rtx-ai-pc/

MLX (Apple): https://ml-explore.github.io/mlx/

PhantomByte Articles:

- "I Cut $900/Month From My Cloud Bill By Going Local"
- "Sovereign AI: Why Your Models Should Run On Your Hardware"
- articles.phantom-byte.com

END OF BLUEPRINT – REVISED & EXPANDED EDITION

Built by PhantomByte – Code from the shadows.

Questions? hello@phantom-byte.com

Ready to go sovereign?

More guides, tools, and tutorials at:

articles.phantom-byte.com